

同济大学计算机系

计算机组成原理实验报告



学 号 2454307

姓 名 汤皓宇

专 业 信息安全

授课老师 郭玉臣

一、实验内容

汇编实现斐波那契数列计算，冒泡排序，Booth 乘法

二、硬件逻辑图

无

三、模块建模

1. 斐波那契数列

通过一个循环结构实现了斐波那契数列的迭代计算，其核心逻辑是首先将寄存器 \$2 和 \$3 分别初始化为数列的前两个项 (0 和 1)，并将计数器 \$4 设定为 2 (由初始值 4 减去 2 得出)，随后在 init_loop 循环体中利用 \$1 作为中间变量执行加法操作 addu \$1, \$2, \$3 来生成新项，并通过连续的 move 指令更新数值，使得 \$2 始终存储前一项而 \$3 存储当前项，每轮循环后通过 subiu 递减计数器并在其不为零时跳转回循环起点，最终在循环结束时使寄存器 \$3 存储斐波那契数列中对应位置的值。

```
.text
    move $2, $0
    li $3, 1
    li $4, 4
    subi $4, $4, 2
init_loop:
    addu $1, $2, $3
    move $2, $3
    move $3, $1
    subiu $4, $4, 1
    bnez $4, init_loop
```

2. 冒泡排序

利用 outer_loop 循环不断遍历这组数据，在每一轮中通过 slt 指令比较相邻两个寄存器的大小，若发现前者大于后者则利用临时寄存器 \$t1 进行数值交换，并使用标志位寄存器 \$t9 记录当前轮次是否发生了交换，直到整个序列完全有序 (即某一轮遍历中没有发生任何交换，\$t9 保持为 0) 时跳出循环并结束程序。

```
.text
main:
    addiu $2, $0, 25
    addiu $3, $0, 7
    addiu $4, $0, 64
    addiu $5, $0, 12
    addiu $6, $0, 3
outer_loop:
    addiu $t9, $0, 0
```

```

        slt $t0, $3, $2
        beq $t0, $0, skip1
        add $t1, $2, $0
        add $2, $3, $0
        add $3, $t1, $0
        addiu $t9, $0, 1
skip1:
        slt $t0, $4, $3
        beq $t0, $0, skip2
        add $t1, $3, $0
        add $3, $4, $0
        add $4, $t1, $0
        addiu $t9, $0, 1
skip2:
        slt $t0, $5, $4
        beq $t0, $0, skip3
        add $t1, $4, $0
        add $4, $5, $0
        add $5, $t1, $0
        addiu $t9, $0, 1
skip3:
        slt $t0, $6, $5
        beq $t0, $0, skip4
        add $t1, $5, $0
        add $5, $6, $0
        add $6, $t1, $0
        addiu $t9, $0, 1
skip4:
        bne $t9, $0, outer_loop
end:
        li $v0, 10

```

3. Booth 乘法

在 `booth_multiply` 子程序中利用寄存器 `$s2` 和 `$s3` 分别存储 64 位乘积的高位和低位，并在 32 次循环中根据乘数最低位与上一次移位出的辅助位（存储在 `$t0`）的组合情况（即 01、10 或相同位）来决定对高位部分进行加、减被乘数或直接移位，每轮计算结束后通过算术右移（`sra`）和逻辑右移（`srl`）相结合的方式将高低位整体右移一位，从而确保能够正确处理补码形式的有符号数相乘，最后将得到的 64 位结果通过 `$v0` 和 `$v1` 返回并输出。

```

.data
msg1: .asciiz "Multiplicand: "
msg2: .asciiz "Multiplier: "
msg3: .asciiz "Product (high 32 bits): "
msg4: .asciiz "Product (low 32 bits): "
newline: .asciiz "\n"

```

```
.text
.globl main
main:
li $t0, 7
li $t1, -3
move $a0, $t0
move $a1, $t1
jal booth_multiply
move $s0, $v0
move $s1, $v1
li $v0, 4
la $a0, msg1
syscall
li $v0, 1
move $a0, $t0
syscall
li $v0, 4
la $a0, newline
syscall
li $v0, 4
la $a0, msg2
syscall
li $v0, 1
move $a0, $t1
syscall
li $v0, 4
la $a0, newline
syscall
li $v0, 4
la $a0, msg3
syscall
li $v0, 1
move $a0, $s0
syscall
li $v0, 4
la $a0, newline
syscall
li $v0, 4
la $a0, msg4
syscall
li $v0, 1
move $a0, $s1
syscall
li $v0, 4
```

```

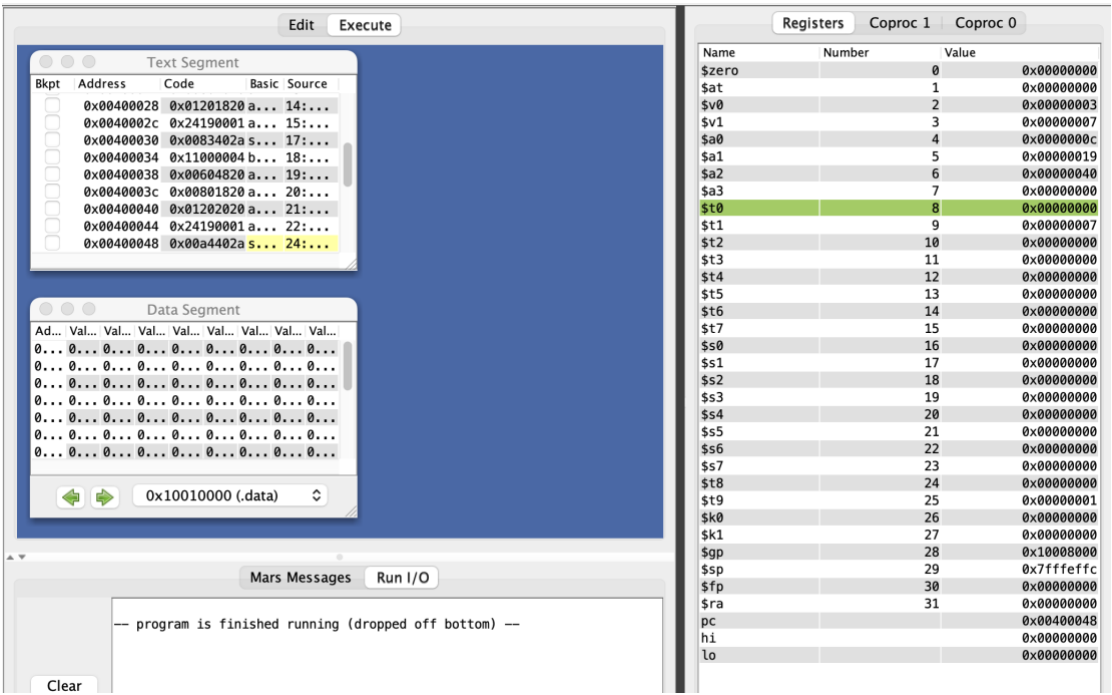
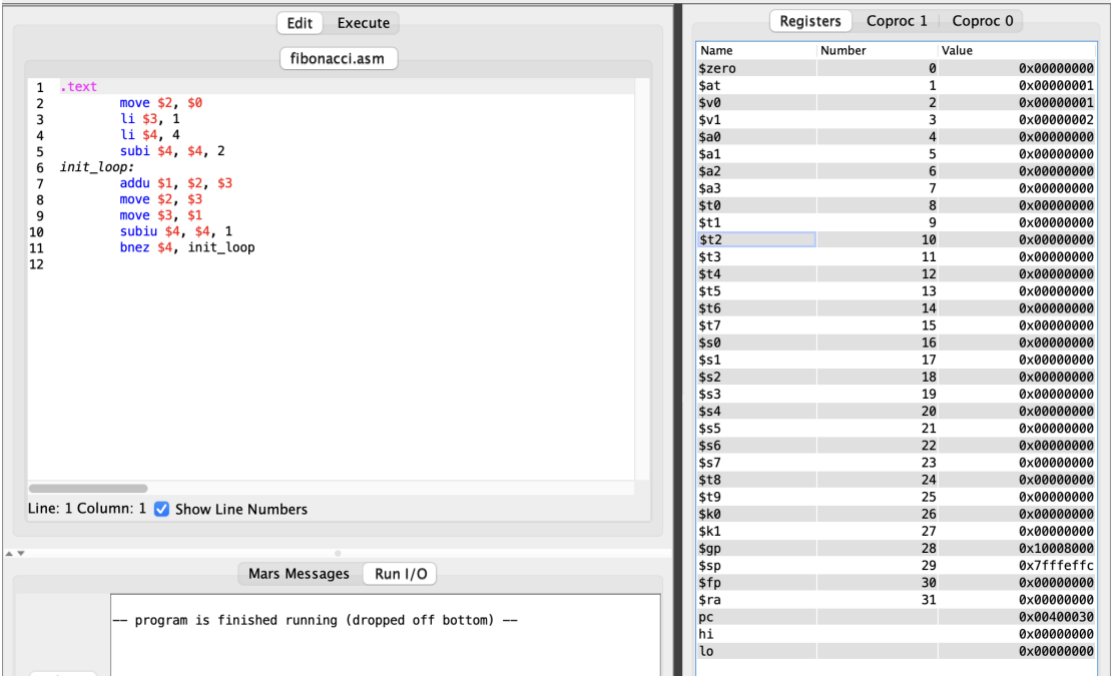
la $a0, newline
syscall
li $v0, 10
syscall
booth_multiply:
addi $sp, $sp, -20
sw $ra, 16($sp)
sw $s0, 12($sp)
sw $s1, 8($sp)
sw $s2, 4($sp)
sw $s3, 0($sp)
move $s0, $a0
move $s1, $a1
li $s2, 0
move $s3, $s1
li $t0, 0
li $t1, 32
booth_loop:
andi $t2, $s3, 1
beq $t2, $t0, no_add_sub
beq $t2, $0, add_mcand
sub $s2, $s2, $s0
j no_add_sub
add_mcand:
add $s2, $s2, $s0
no_add_sub:
andi $t3, $s2, 1
sra $s2, $s2, 1
srl $s3, $s3, 1
sll $t3, $t3, 31
or $s3, $s3, $t3
move $t0, $t2
addi $t1, $t1, -1
bnez $t1, booth_loop
move $v0, $s2
move $v1, $s3
lw $s3, 0($sp)
lw $s2, 4($sp)
lw $s1, 8($sp)
lw $s0, 12($sp)
lw $ra, 16($sp)
addi $sp, $sp, 20
jr $ra

```

四、测试模块建模

无

五、实验结果



EditExecute

Text Segment

Bkpt	Address	Code	Basic	Source
	0x00400000	0x24080007 a...	10:...	
	0x00400004	0x24090003 a...	11:...	
	0x00400008	0x00082021 a...	12:...	
	0x0040000c	0x00092821 a...	13:...	
	0x00400010	0x0c100035 j...	14:...	
	0x00400014	0x00028021 a...	15:...	
	0x00400018	0x00038821 a...	16:...	
	0x0040001c	0x24020004 a...	17:...	
	0x00400020	0x3c011001 l...	18:...	

Data Segment

Ad...	Val...	Val...	Val...	Val...	Val...	Val...	Val...
0...	0...	0...	0...	0...	0...	0...	0...
0...	0...	0...	0...	0...	0...	0...	0...
0...	0...	0...	0...	0...	0...	0...	0...
0...	0...	0...	0...	0...	0...	0...	0...
0...	0...	0...	0...	0...	0...	0...	0...
0...	0...	0...	0...	0...	0...	0...	0...
0...	0...	0...	0...	0...	0...	0...	0...
0...	0...	0...	0...	0...	0...	0...	0...

0x10010000 (.data)

Mars MessagesRun I/O

-- program is finished running (dropped off bottom) --
Multiplicand: 0

RegistersCoproccoproc 0

Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x10010000
\$v0	2	0x0000000a
\$v1	3	0x00000015
\$a0	4	0x1001004d
\$a1	5	0x00000003
\$a2	6	0x00000000
\$a3	7	0x00000000
\$t0	8	0x00000000
\$t1	9	0x00000000
\$t2	10	0x00000000
\$t3	11	0x00000000
\$t4	12	0x00000000
\$t5	13	0x00000000
\$t6	14	0x00000000
\$t7	15	0x00000000
\$s0	16	0x00000000
\$s1	17	0x00000015
\$s2	18	0x00000000
\$s3	19	0x00000000
\$s4	20	0x00000000
\$s5	21	0x00000000
\$s6	22	0x00000000
\$s7	23	0x00000000
\$t8	24	0x00000000
\$t9	25	0x00000000
\$k0	26	0x00000000
\$k1	27	0x00000000
\$gp	28	0x10000000
\$sp	29	0x7fffeffc
\$fp	30	0x00000000
\$ra	31	0x00400014
pc		0x004000d4
hi		0x00000000
lo		0x00000000